Noname manuscript No.
(will be inserted by the editor)

ReRAPTOR: Reverse-Objective RAPTOR for Time-Budgeted Value-Maximizing Transit Itinerary Planning

Bin Zhou · Guangxian Li · Teruki Ogo · Yutaka Arakawa · Tsunenori Mine

Received: date / Accepted: date

Abstract Public-transit routing is usually formulated as an earliest-arrival problem, but leisure exploration and time-budgeted outing design require timetable-feasible routes that maximize itinerary value rather than merely minimize travel time. This paper studies time-budgeted value-maximizing transit itinerary planning on timetable-based transit schedules and proposes ReRAPTOR, a reverse-objective RAPTOR-style label-setting method that preserves timetable-feasible expansion while ranking partial routes by a value-aware surrogate score. The framework supports pluggable objective modules and uses three pruning layers: Beam pruning for width control, Dominance pruning for local label filtering, and Negative pruning for suppressing semantically weak partial routes. We evaluate the method on Ekispert-provided transit schedule data for Fukuoka under a seeded stop-reward benchmark and compare it with NSGA-II, ILS+2opt, VNS, SA, McRAPTOR, Pure RAPTOR, and Greedy. ReRAPTOR achieves the highest overall average stop score (826.90), improving on NSGA-II by 8.71%, while remaining 6.57× faster on average (4165.67 ms vs 27351.18 ms). A supplementary balance analysis further shows that ReRAPTOR is the closest method to the score–time ideal point, with query-level ideal distance 0.1672 versus 0.6292 for NSGA-II. Ablation results show that Beam and Dominance are the two core scalability layers, whereas Negative pruning mainly improves semantic alignment in theme-constrained tasks. These results indicate that ReRAPTOR provides a strong quality–speed trade-off for value-oriented transit itinerary planning.

Keywords Transit itinerary planning · Transit schedule data · RAPTOR · Time-budgeted route planning · Multi-objective search

Bin Zhou
Graduate School of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: zhou.bin.883@s.kyushu-u.ac.jp

Guangxian Li
Graduate School of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: li.guangxian.021@s.kyushu-u.ac.jp

Teruki Ogo
Faculty of Information Science and Engineering, Kyushu Sangyo University, Fukuoka, Japan
E-mail: k25rm021@st.kyusan-u.ac.jp

Yutaka Arakawa
Faculty of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: arakawa@ait.kyushu-u.ac.jp

Tsunenori Mine
Faculty of Information Science and Electrical Engineering, Kyushu University, Fukuoka, Japan
E-mail: mine@ait.kyushu-u.ac.jp

1 Introduction

Public-transit route planning is traditionally formulated as a timetable-feasible earliest-arrival problem [2,6,8,17]. That formulation is appropriate for commuters, but it does not capture the needs of leisure travelers, city explorers, or users who want to spend a fixed amount of time on transit while collecting as much route value as possible. In these settings, a good itinerary is not defined only by arrival time. It may also depend on ride duration, the number of distinct stops visited, spatial spread, and the thematic or semantic quality of the route. The problem is therefore closer to time-budgeted itinerary design than to classical shortest-path search [16,31,33].

The main technical difficulty is the combination of timetable feasibility and value-oriented search. Every

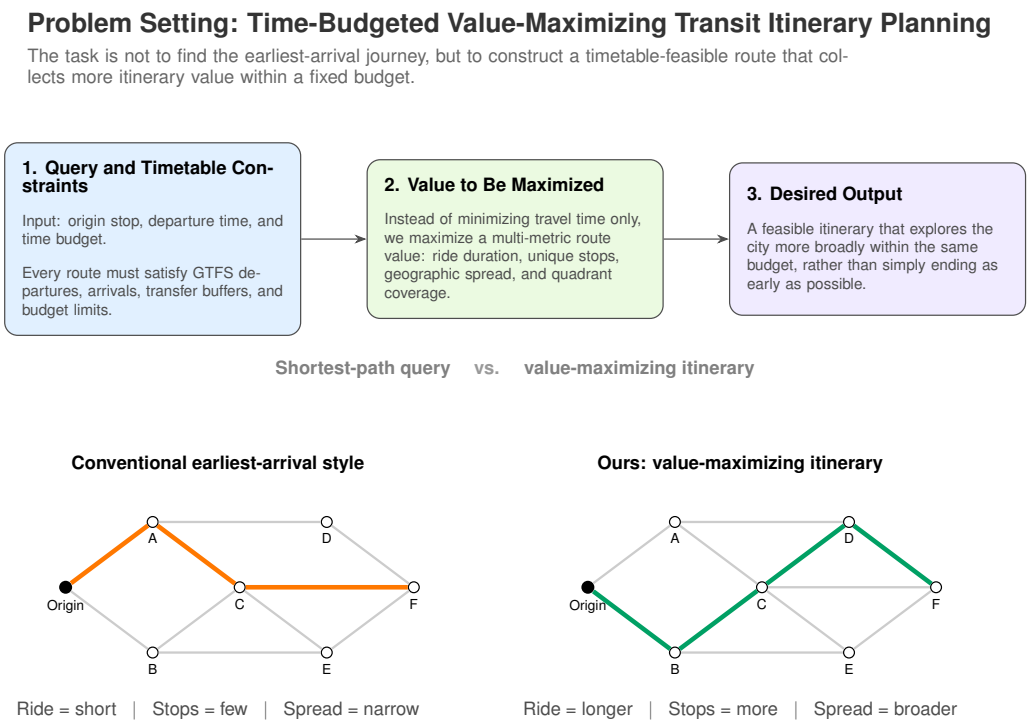


Fig. 1 Problem overview of time-budgeted value-maximizing transit itinerary planning. Unlike conventional earliest-arrival routing, the goal is to keep timetable feasibility while maximizing multi-dimensional itinerary value within a fixed travel budget.

candidate route must respect scheduled departures, arrivals, and transfer constraints, while the search objective depends on accumulated route structure rather than on arrival time alone. Once stop diversity, spatial expansion, revisit patterns, or task-specific stop rewards are incorporated, the number of plausible partial routes grows quickly. An overly aggressive compression strategy removes information needed for value maximization, whereas an unpruned search becomes computationally unstable.

This paper addresses that gap with ReRAPTOR, a reverse-objective RAPTOR-style search over timetable-feasible transit stop schedules [8,37,38]. Instead of propagating labels to minimize arrival time, ReRAPTOR preserves the timetable-feasible expansion logic of RAPTOR while ranking partial routes by a task-dependent value surrogate. The framework is generic: the search backbone is shared, while the scoring and acceptance modules can be instantiated for geometry-aware exploration or for the seeded stop-reward task used in our main experiments. To keep the search tractable, ReRAPTOR combines three pruning layers: Beam pruning for width control, Dominance pruning for local label filtering, and Negative pruning for rejecting semantically weak or clearly low-quality partial routes. Figure 1 illustrates this shift from earliest-arrival routing to time-budgeted value collection.

We evaluate the method on Ekispert-provided transit schedule data for Fukuoka through four experiment groups. Experiment 1 compares ReRAPTOR with NSGA-II [5], ILS+2opt [21], VNS [24], SA [19], McRAPTOR and Pure RAPTOR [8], and Greedy under a seeded stop-reward benchmark. ReRAPTOR achieves the highest overall average stop score (826.90) and is 6.57× faster than NSGA-II on average (4165.67 ms vs 27351.18 ms). Experiment 2 shows that Beam and Dominance are the two core scalability layers: removing Dominance makes the method 2.91× slower, removing Beam makes it 9.53× slower, and removing both causes severe label explosion and cap hits. Experiment 3 further shows that ReRAPTOR is the closest method to the score–time ideal point, with query-level ideal distance 0.1672 compared with 0.6292 for NSGA-II. Experiment 4 complements the aggregate numbers with

route-level cases, including both strong success cases and an explicit counterexample.

The contributions of this paper are fourfold. First, we formulate time-budgeted value-maximizing transit itinerary planning as a timetable-feasible search problem with pluggable objective and acceptance modules, so the same search backbone can support geometry-aware and seeded stop-reward tasks. Second, we propose ReRAPTOR, a reverse-objective RAPTOR-style label-setting algorithm that keeps route history, spatial coverage, and task-specific reward state inside timetable expansion rather than scoring routes only after construction. Third, we show that the proposed three-layer pruning design has distinct roles: Beam and Dominance act as the main search-space control layers, whereas Negative pruning mainly improves semantic alignment in constrained tasks. Fourth, experiments on Ekispert-provided Fukuoka transit schedules show that ReRAPTOR achieves the strongest overall quality–speed profile among the compared baselines while remaining explicit about failure cases.

The remainder of the paper is organized as follows. Section 2 reviews related work on tourist trip design, public-transit routing, and multi-objective search. Section 3 formalizes the problem. Section 4 presents ReRAPTOR and its mathematical abstraction. Section 5 reports experimental results, including algorithm comparison, ablation, balance analysis, and a route case study. Finally, Sections 6 and 7 discuss implications and conclude the paper.

2 Related Work

Timetable-based public-transit routing has been studied extensively, with RAPTOR being a representative round-based framework for computing efficient earliest-arrival and Pareto-feasible journeys without heavy preprocessing [8]. Subsequent work improves exact or near-exact search through trip-based bicriterion routing, transfer-aware Pareto exploration, restricted Pareto sets, hyper-partitioning, hub-label acceleration, and transfer connection databases [1,2,6,7,17,27]. Other studies consider public-transport routing objectives beyond travel time, including fare or price optimization and multi-objective Monte-Carlo tree search [11,36]. However, these methods mainly target earliest-arrival, transfer-minimizing, fare-aware, or Pareto-routing queries, rather than itinerary design under a fixed budget where route value depends on accumulated route structure and exploration history.

GTFS has also become a common substrate for public-transport analysis and algorithm design. Prior work has used GTFS for efficient transit analysis, retrospective timetable reconstruction, time-geographic analysis, and database querying of public-transport networks [14,25,37,38]. Public transport network datasets have further been assembled to support algorithmic evaluation across cities [20]. These studies show that GTFS is a practical and reproducible data source, but they do not directly address value-maximizing itinerary search on timetable-feasible stop schedules.

The itinerary-design side of our problem is closely related to the tourist trip design problem and the broader orienteering family. Surveys show that budget-constrained profit collection, team orienteering, time windows, and preference-aware route construction are central themes in this literature [16,31]. Representative works study personalized tourist trip design, trip-planning functionalities, time-dependent team orienteering for tourist routing, and social-sensing-based personalization [13,26,28,32,33]. Recent variants further consider green or fuzzy objectives, hotel selection, capacity constraints, mandatory visits, GRASP-based multimodal optimization, and incremental local search [3,10,15,18,22,23,29,30,34]. Despite their richness, these formulations often assume free movement between points of interest or simplified travel-time models, rather than operating directly on timetable-feasible transit stop schedules.

More generally, multi-objective trip planning and shortest-path research has studied ranking, preference integration, and dominance filtering in route search [4, 9,12,35]. On the optimization side, canonical general-purpose baselines for value-guided search include NSGA-II, iterated local search, variable neighborhood search, and simulated annealing [5,19,21,24]. Our work differs from both classical public-transit routing and tourist-trip design in one specific way: it embeds a pluggable value module inside a RAPTOR-style timetable search, so that itinerary value is optimized during feasible schedule expansion rather than only after constructing a route.

3 Problem Formulation

We model the transit timetable as a temporal directed graph [25,37,38] $G = (\mathcal{S}, \mathcal{E})$, where each stop $s \in \mathcal{S}$ is a station identifier and each edge $e \in \mathcal{E}$ is a feasible timetable segment $e = (u, v, \tau^{\mathrm{dep}}(e), \tau^{\mathrm{arr}}(e), \ell(e), d(e))$. Here u and v are origin and destination stops, τ^{dep} and τ^{arr} are departure/arrival times, $\ell(e)$ is the transit line ID, and $d(e)$ is the segment distance. Given a query $q = (o, t_0, B)$ with origin o, departure time t_0, and time budget B, the task is to construct a timetable-feasible itinerary $r = (e_1, \ldots, e_m)$ that maximizes route value.

A route is feasible if it starts from the query origin, respects stop continuity, satisfies transfer-buffer constraints, and finishes within the budget:

$$\begin{aligned} u(e_1) &= o, \\ \tau^{\mathrm{dep}}(e_1) &\geq t_0, \\ v(e_i) &= u(e_{i+1}), \\ \tau^{\mathrm{dep}}(e_{i+1}) &\geq \tau^{\mathrm{arr}}(e_i) + \delta(e_i, e_{i+1}), \\ \tau^{\mathrm{arr}}(e_m) &\leq t_0 + B, \end{aligned} \tag{1}$$

where $\delta(e_i, e_{i+1}) = 0$ for same-trip continuation and $\delta(e_i, e_{i+1}) = \Delta_{\mathrm{tr}}$ otherwise.

Instead of fixing one route-value function for all tasks, we define a generic objective module parameterized by θ:

$$\max_{r \in \mathcal{R}(q)} F_\theta(r) \qquad \text{s.t. } g_\theta(r) = 1, \tag{2}$$

where $\mathcal{R}(q)$ is the set of timetable-feasible routes under Eq. 1, F_θ is a scenario-specific route-value function, and g_θ is an optional acceptance predicate. This separation is important in our setting because the search backbone remains unchanged across experiments, while only the value module and acceptance logic are instantiated differently.

The default geometry-aware instance in the original formulation evaluates four positive criteria: $f_1(r) =$ ride duration, $f_2(r) =$ number of unique visited stops, $f_3(r) =$ convex-hull area of visited stop coordinates, and $f_4(r) =$ quadrant coverage around the city center. Using normalizers $c_1 = 480$, $c_2 = 50$, $c_3 = 100$, and $c_4 = 4$, we define $\tilde{f}_j(r) = f_j(r)/c_j$ and optimize

$$F_{\mathrm{geo}}(r) = \sum_{j=1}^{4} \alpha_j \tilde{f}_j(r), \tag{3}$$

The default weights are 0.40, 0.25, 0.20, and 0.15. Here $c_1 = 480$ and $c_4 = 4$ come from the problem itself: the 8-hour budget ceiling and the maximum of four covered quadrants. By contrast, $c_2 = 50$ and $c_3 = 100$ are reference scales that keep the stop-count and hull-area terms comparable. The weight vector is not re-tuned in this paper. We keep the balanced geometry-aware profile from the original Fukuoka system, with ride duration as the main term, stop diversity as the second term, and spatial spread as supporting evidence.

Experiments 1–3 use a seeded stop-reward instantiation. Each stop $s \in \mathcal{S}$ receives a reward $w_\omega(s)$ drawn once from a random seed ω, and each unique non-origin stop incurs a dwell time η. Let $V^+(r)$ denote the set of unique visited stops excluding the origin, and let $T(r)$ be the elapsed timetable travel time of route r. The effective task time is

$$T_\eta(r) = T(r) + \eta\,|V^+(r)|. \tag{4}$$

The seeded stop-reward objective is then

$$F_{\omega,\eta}(r) = \sum_{s \in V^+(r)} w_\omega(s), \qquad g_{\omega,\eta}(r) = \mathbf{1}[T_\eta(r) \leq B]\,. \tag{5}$$

In other words, the main experiments no longer optimize hull area directly; they optimize collected stop reward under a fixed effective-time budget, while the same timetable-feasible search structure and pruning framework are retained.

4 Proposed Method

4.1 Overall Framework

ReRAPTOR is a RAPTOR-style label-setting search over timetable-feasible transit stop schedules with pluggable objective modules and a three-layer pruning mechanism [8,17]. Unlike earliest-arrival transit routing, the search state explicitly stores route history and exploration statistics. Partial itineraries can therefore be ranked by both budgeted utility and route structure. At a high level, the framework has five stages:

1. load transit stop schedules and query parameters;
2. initialize a base label at the origin stop;
3. iteratively expand timetable-feasible outgoing edges from marked stops;
4. evaluate each new partial itinerary with an objective-specific surrogate score and prune it with the three-layer filter;
5. return the highest-scored accepted label as the final route.

Figure 2 highlights the separation between the shared search backbone and the pluggable scoring/acceptance modules. This separation lets us study algorithmic comparison (Exp1), pruning effects (Exp2), and balance behavior (Exp3) without redesigning the search engine.

4.2 Core Algorithm

Each partial route is represented by a label

$$\lambda = \left(a, \rho, d, V, q, \Pi, c^{\mathrm{stop}}, c^{\mathrm{line}}, n^{\mathrm{tr}}, \psi\right), \tag{6}$$

where a is the current arrival time, ρ is accumulated ride time, d is traveled distance, V is the set of visited stops, q is a bit-mask encoding covered quadrants, Π is the ordered leg sequence, c^{stop} and c^{line} are visit counters, and n^{tr} is the number of transfers. The additional term ψ stores objective-specific accumulators. In the stop-reward task used for Experiments 1–3, ψ includes at

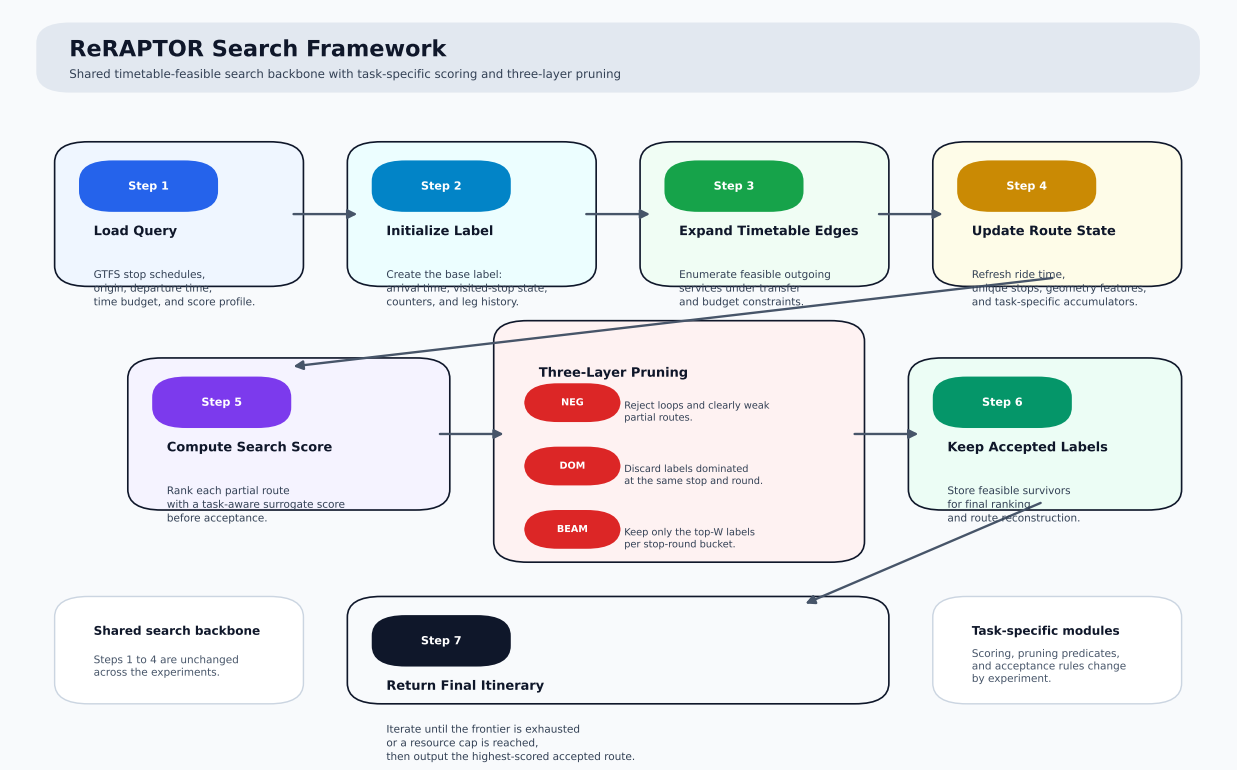


Fig. 2 Overall framework of the proposed time-budgeted itinerary planner. The core search remains unchanged across experiments; only score weights or optional acceptance predicates are varied.

least the collected reward total σ and the number of unique scored stops m.

Given a feasible edge $e = (u, v, \tau^{\mathrm{dep}}, \tau^{\mathrm{arr}}, \ell, d_e)$, the extension operator updates the label by

$$\begin{aligned} a' &= \tau^{\mathrm{arr}}(e), \\ \rho' &= \rho + \left(\tau^{\mathrm{arr}}(e) - \tau^{\mathrm{dep}}(e)\right), \\ d' &= d + d_e, \qquad V' = V \cup \{v\}, \\ q' &= q \vee q(v), \qquad \Pi' = \Pi \oplus e, \\ \psi' &= U_\theta(\psi, \lambda, e), \end{aligned} \tag{7}$$

while the stop and line counters are incremented on the destination stop and on new line boardings, respectively. Here $q(v)$ denotes the quadrant indicator of stop v, and U_θ is an objective-specific state update. For the stop-reward instantiation, U_θ adds $w_\omega(v)$ only when v is a newly scored stop and increments m accordingly. This state design makes geometric coverage, revisit patterns, transfer behavior, and task-specific reward accumulation available during search rather than only after route construction.

The search surrogate used for label ranking is written in generic form as

$$S_\theta(\lambda) = \Phi_\theta(\lambda, \psi(\lambda)) - P(\lambda), \tag{8}$$

where Φ_θ is the task-specific positive utility module and $P(\lambda)$ is the negative-pruning term. The negative penalty is

$$\begin{aligned} P(\lambda) = 50000\,\mathbf{1}\Big[\max_s c_\lambda^{\mathrm{stop}}(s) > 2\Big] \\ + 20000\,\mathbf{1}[\rho(\lambda)/|V(\lambda)| < 2] \\ + P_\theta(\lambda), \end{aligned} \tag{9}$$

where P_θ is an optional scenario-specific penalty used only when theme or scenario constraints are explicitly instantiated. The threshold $\max_s c_\lambda^{\mathrm{stop}}(s) > 2$ suppresses clear revisit loops, while $\rho(\lambda)/|V(\lambda)| < 2$ suppresses chains of unrealistically short hops; the coefficients 50000 and 20000 are safety-side heuristic defaults inherited from the early implementation, not values tuned from the reported experiments.

For the default geometry-aware instance,

$$\begin{aligned} S_{\mathrm{geo}}(\lambda) = \beta_1 \rho(\lambda) + \beta_2 |V(\lambda)| \\ + \beta_3 A(V(\lambda)) + \beta_4 Q(V(\lambda)) \\ - P(\lambda), \end{aligned} \tag{10}$$

where $A(\cdot)$ is the convex-hull area of visited stops after planar projection, $Q(\cdot) \in \{1, 2, 3, 4\}$ is the number of covered quadrants, and the default coefficients are $(\beta_1, \beta_2, \beta_3, \beta_4) = (1.5, 10, 50, 100)$. These β values are fixed geometry-aware heuristic defaults from the original system. Their ordering reflects the intended search bias:

broaden directional and spatial coverage before merely extending ride time.

For the stop-reward task used in Experiments 1–3, the search score is lexicographically biased toward collected reward and only uses time-based terms as tie-breakers:

$$\begin{aligned} S_{\omega,\eta}(\lambda) = {} & 10000\,\sigma(\lambda) + 5\ \mathrm{SPH}(\lambda) - 0.1\,T_\eta(\lambda) \\ & - 50000\ \max(0, T_\eta(\lambda) - B) - P(\lambda), \end{aligned} \tag{11}$$

where $\sigma(\lambda)$ is the collected stop-reward total, $T_\eta(\lambda)$ is the effective time of the partial route, and $\mathrm{SPH}(\lambda) = 60\,\sigma(\lambda)/T_\eta(\lambda)$ is the score-per-hour tie-breaker. The large coefficient on $\sigma(\lambda)$ ensures that even a small gain in collected reward dominates secondary terms, which matches the seeded stop-reward evaluation protocol. The constants $10000, 5, 0.1$, and 50000 are design defaults for implementing the intended lexicographic priority "reward $\gg$ efficiency $\gg$ time"; the factor 60 in SPH is only a minute-to-hour unit conversion.

After scoring, each candidate is filtered by two additional pruning stages. First, dominance pruning discards labels that are clearly weaker than an existing label at the same stop and round. As in multi-objective shortest-path and multicriteria transit-routing literature, this step removes labels that are unlikely to produce better final outcomes [9,17]. In generic form, λ^a dominates λ^b if

$$\lambda^a \succeq_\theta \lambda^b \iff D_\theta(\lambda^a, \lambda^b) = 1. \tag{12}$$

For the default geometry-aware instance, we use the heuristic rule

$$\begin{aligned} \lambda^a \succeq_{\mathrm{geo}} \lambda^b \iff {} & S_{\mathrm{geo}}(\lambda^a) \geq S_{\mathrm{geo}}(\lambda^b) \\ & \wedge\ \rho(\lambda^a) \geq 0.95\,\rho(\lambda^b) \\ & \wedge\ A(V(\lambda^a)) \geq A(V(\lambda^b)). \end{aligned} \tag{13}$$

For the stop-reward task, we use a tighter seeded stop-reward rule based on reward, effective time, and an optimistic upper bound:

$$\begin{aligned} \lambda^a \succeq_{\omega,\eta} \lambda^b \iff {} & \sigma(\lambda^a) \geq \sigma(\lambda^b) \\ & \wedge\ T_\eta(\lambda^a) \leq T_\eta(\lambda^b) \\ & \wedge\ U_{\omega,\eta}(\lambda^a) \geq U_{\omega,\eta}(\lambda^b), \end{aligned} \tag{14}$$

with at least one strict inequality. Here $U_{\omega,\eta}(\lambda)$ is an optimistic upper bound on the final stop reward, computed from the current reward total plus the best uncollected stop scores that can still fit into the remaining effective-time budget. Second, beam pruning keeps only the best W labels in each stop-round bucket:

$$\begin{aligned} \mathcal{L}_{k+1}(v) = {} & \mathrm{TopW}_{S_\theta}\Big(\\ & \big\{\lambda' \,\big|\, \lambda \in \mathcal{L}_k(u), \\ & \quad e \in \mathrm{Next}(u, a_\lambda), \\ & \quad \lambda' = \mathrm{Extend}(\lambda, e)\big\}\Big). \end{aligned} \tag{15}$$

Here $D_\theta(\cdot,\cdot)$ denotes the task-specific dominance predicate, TopW_{S_θ} returns the highest-scored W labels under S_θ, and $\mathrm{Next}(u, a_\lambda)$ denotes the feasible outgoing timetable edges from stop u no earlier than arrival time a_λ. In our experiments, $W = 50$ is the default setting; this is a fixed practical beam-width preset rather than a value optimized in the paper. The route returned at the end of search is the highest-scored accepted label, where acceptance requires at least two legs and may optionally include an experiment-specific predicate.

4.3 Complexity and Implementation

Let R be the maximum number of search rounds, W the beam width, and $C_k(v)$ the number of candidate timetable edges returned by $\mathrm{Next}(v,\cdot)$ at round k. Because beam pruning bounds each stop-round bucket by W, the practical number of expansions is

$$O\left(\sum_{k=0}^{R-1} \sum_{v \in \mathcal{M}_k} W\, C_k(v)\right), \tag{16}$$

where $\mathcal{M}_k$ is the marked-stop set at round k. Each insertion additionally performs $O(W)$ dominance checks and $O(W \log W)$ beam maintenance, which yields a conservative upper bound of $O(R|\mathcal{S}|CW(W + \log W))$ when $C_k(v)$ is approximated by an average candidate count C. The memory footprint is $O(|\mathcal{S}|W)$ per round because each bucket stores at most W labels.

The implementation directly expands labels over stop schedules through the `_next_edges(...)` method of the planner object, rather than reconstructing route tables. This detail is important in our dataset because some `trip_id` values are not globally unique, and route-table reconstruction may drop valid services. To reduce repeated geometry computation, metric tuples are memoized by a label signature containing arrival time, ride time, visited stops, and leg descriptors. The default implementation uses a 120-minute lookahead window and $R = \min(220, \max(40, \lfloor B/2 \rfloor + 20))$, while time-out and label-cap guards are retained for ablation and scalability studies. These implementation constants are fixed defaults used to scale search depth with budget and to keep runtime bounded; they are not claimed as optimally tuned values.

Table 1 Experiment 1 setup (algorithm comparison).

Item	Setting
Transit data	Fukuoka transit timetable segments derived from Ekispert-provided schedule data: 304,344 records, 1,032 stops, 302 lines.
Query grid	10 origins × 3 departure times (`07:00`, `10:00`, `14:00`) × 4 budgets (`2h`, `4h`, `6h`, `8h`).
Repetitions	5 random stop-score seeds per query.
Compared methods	Ours, NSGA-II [5], ILS+2opt [21], VNS [24], SA [19], McRAPTOR [8], Pure RAPTOR [8], and Greedy.
Total runs	600 task rows per algorithm; 4,800 paired task comparisons overall. Before aggregation, stochastic baselines produce 16,800 raw executions.
Primary metrics	`stop_score_total`, `elapsed_ms`, `score_per_hour`, and `effective_budget_feasible`.
Aggregation	Query-level mean over score seeds: 30 queries per budget and 120 overall.
Pairing unit	Task counts use origin, departure time, budget, and score seed; the main table uses base-query means.

5 Experiments

5.1 Experiment 1: Comprehensive Algorithm Comparison

5.1.1 Objective

Experiment 1 evaluates whether ReRAPTOR provides better overall route quality than representative baselines after reformulating the benchmark to a seeded stop-reward task. Instead of relying on hull area as the primary benchmark outcome, each stop is assigned a random score and the task is to maximize the collected stop score within a fixed time budget. The three evaluation questions are:

- **RQ1 (quality):** Does Ours improve the primary task metric, `stop_score_total`?
- **RQ2 (efficiency):** Does Ours achieve high score while remaining fast on many queries?
- **RQ3 (robustness):** Are the gains stable across budgets and random stop-score seeds?

5.1.2 Setup and Protocol

Table 1 reports the setup. All compared methods share the same query set, time budgets, and underlying transit schedule dataset version.

5.1.3 Main Results

Table 2 gives the overall (2h+4h+6h+8h) query-level means. ReRAPTOR achieves the highest average stop score (826.90), exceeding NSGA-II (760.66) by +8.71%, while also being 6.57× faster overall (4165.67 ms vs 27351.18 ms). The main claim of Experiment 1 is therefore not only "best average score", but more importantly that Ours achieves *high score and fast response on many queries*.

Table 2 Overall summary of Experiment 1 (query-level means, 120 queries).

Method	Stop Score	Elapsed (ms)	Score / hour	Avg. rank
Ours (Full)	826.90	4165.67	193.02	1.94
NSGA-II	760.66	27351.18	170.40	2.66
ILS+2opt	697.46	518.79	157.10	3.33
VNS	645.00	169.31	148.13	4.30
McRAPTOR	546.70	3684.86	149.33	5.04
SA	589.46	130.80	137.25	5.39
Greedy (L=3)	416.99	3.05	101.27	6.99
Pure RAPTOR	320.31	56483.31	192.36	6.34

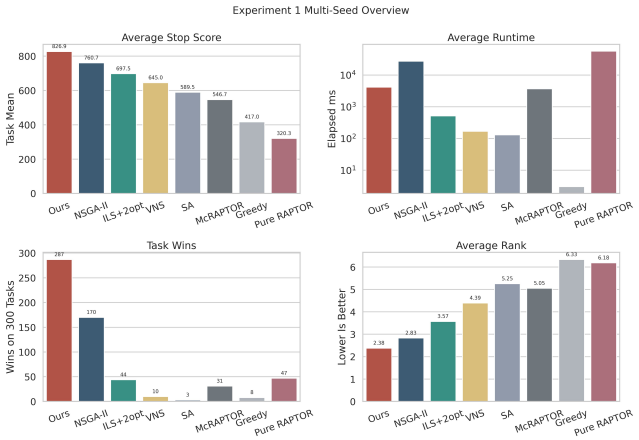


Fig. 3 Overview of Experiment 1 under the score-guided multi-seed benchmark. The central message is that Ours most often lies in the high-score and low-runtime region, while also ranking first in average stop score.

At the finer task level, Ours beats NSGA-II in 368/600 comparisons. After averaging over stop-score seeds, Ours still achieves a higher stop score than NSGA-II in 82/120 base queries. Figure 3 visualizes this combined view and summarizes the main result of Experiment 1.

5.1.4 Budget-Sliced Analysis

To avoid mixed-budget interpretation bias, Table 3 reports 2h, 4h, 6h, and 8h separately. Ours remains stronger than NSGA-II in both settings:

- 2h: stop score 350.37 vs 322.62, while runtime is 83.95 ms vs 17026.77 ms.
- 4h: stop score 725.78 vs 624.94, while runtime is 1329.04 ms vs 26629.94 ms.
- 6h: stop score 973.57 vs 900.64, while runtime is 2226.92 ms vs 29981.78 ms.
- 8h: stop score 1257.89 vs 1194.46, while runtime is 13022.77 ms vs 35766.26 ms.

At the task level, Ours outperforms NSGA-II in 69/150, 113/150, 90/150, and 96/150 comparisons for 2h, 4h, 6h, and 8h, respectively. Figure 4 visualizes this budget-sliced progression and complements Table 3.

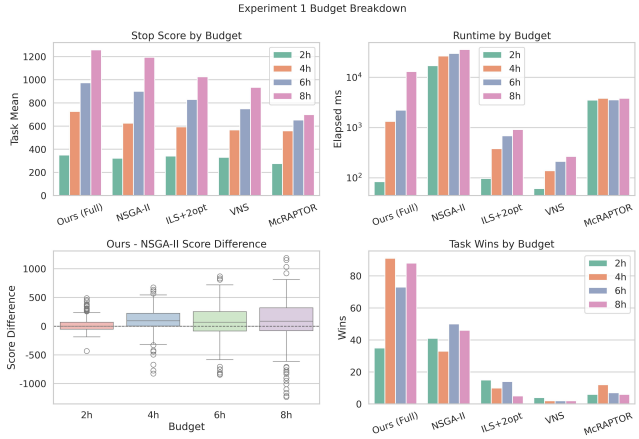


Fig. 4 Budget-sliced overview of Experiment 1. The overall view highlights the global score–runtime pattern, whereas the budget breakdown makes the 2h-to-8h progression explicit.

Table 3 Budget-sliced summary of Experiment 1 (query-level means).

Budget	Method	Stop Score	Elapsed (ms)
2h	Ours (Full)	350.37	83.95
2h	NSGA-II	322.62	17026.77
4h	Ours (Full)	725.78	1329.04
4h	NSGA-II	624.94	26629.94
6h	Ours (Full)	973.57	2226.92
6h	NSGA-II	900.64	29981.78
8h	Ours (Full)	1257.89	13022.77
8h	NSGA-II	1194.46	35766.26

Table 4 Score-seed robustness in Experiment 1 (task-level Ours vs NSGA-II).

Seed	Ours wins	NSGA-II wins	Avg. gap
20260323	75	45	+90.29
20260324	73	43	+107.43
20260325	72	46	+86.32
20260326	78	37	+15.15
20260327	70	47	+32.00

5.1.5 Robustness Across Score Seeds

Table 4 summarizes Ours vs NSGA-II by random stop-score seed. The score gap stays positive for all tested seeds. The weakest seed is `20260326`, but even there the average Ours–NSGA score gap remains +15.15. In addition, all observed zero-score cases of Ours in this run are `effective_budget_feasible = False` edge cases rather than valid-but-poor routes.

The supplementary seed-stability figure is provided in Appendix A (Figure 8).

Table 5 Main ablation summary of Experiment 2 (task-level means over 180 (*origin*, *depart_time*, *budget*, *score_seed*) units).

Method	StopScore	Elapsed (ms)	Labels	CapHit (%)
Ours (Full)	999.27	4668.15	115.4K	0.00
Ours (NoNeg)	996.10	4865.37	119.3K	0.00
Ours (NoDom)	971.91	13589.45	349.9K	0.00
Ours (NoBeam)	912.04	44486.93	1.49M	59.44
Ours (NoBeam-NoDom)	313.79	61694.58	2.81M	100.00
Ours (NoDom-NoNeg)	973.06	12984.17	351.0K	0.00
Ours (NoBeam-NoNeg)	911.56	44700.47	1.51M	60.00
Ours (NoBeam-NoDom-NoNeg)	289.37	20929.99	1.00M	100.00

5.2 Experiment 2: Three-Layer Pruning Ablation and Supporting Evidence

5.2.1 Objective and Research Questions

Experiment 2 isolates the three pruning layers in ReRAPTOR: *Beam pruning* for width control, *Dominance pruning* for local label filtering, and *Negative-score pruning* for rule- or theme-aligned rejection. We answer the following questions:

- **RQ4 (core scalability):** Are Beam and Dominance jointly indispensable for tractable search?
- **RQ5 (budget sensitivity):** How do Beam and Dominance behave at 4h and 8h?
- **RQ6 (semantic alignment):** Does Negative pruning improve route-theme preservation in project-style theme tasks?

5.2.2 Main Ablation Protocol

The main ablation uses the same base query grid as Experiment 1: 10 origins × 3 departure times × 2 budgets (4h, 8h) × 3 stop-score seeds. With 8 algorithmic configurations, this yields 1440 runs in total.

The evaluated variants are `Full`, `NoNeg`, `NoBeam`, `NoDom`, `NoBeam-NoDom`, `NoDom-NoNeg`, `NoBeam-NoNeg`, and `NoBeam-NoDom-NoNeg`. The main metrics are stop score, runtime, label count, and cap-hit rate.

Table 5 shows that `Ours (Full)` has the best combined profile across score, runtime, and search size. `Ours (NoNeg)` is nearly tied on score, which is consistent with the claim that Negative pruning is not the main speedup layer in the generic benchmark. By contrast, removing either Dominance or Beam is clearly expensive. Relative to Full, `NoDom` is 2.91× slower and uses 3.03× more labels, while `NoBeam` is 9.53× slower and uses 12.89× more labels. The newly added `NoBeam-NoDom` variant is the key evidence: once the two main search-space control layers are removed together, the score drops by −685.48, runtime grows to 13.22× Full, and label count grows to 24.36× Full.

Therefore, the updated interpretation of Experiment 2 is that **Beam and Dominance are co-core effi-**

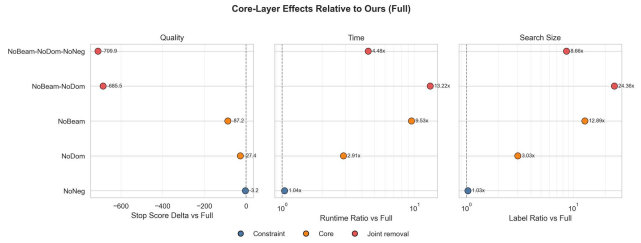


Fig. 5 Relative effects of the main ablation variants with respect to `Ours (Full)`. This figure makes the current interpretation explicit: Beam and Dominance are the two core search-space control layers, while Negative has a role-separated effect.

Table 6 Budget-sliced comparison of `Full` and `NoBeam`.

Budget	Method	StopScore	Elapsed (ms)	CapHit (%)
4h	Ours (Full)	737.04	771.47	0.00
4h	Ours (NoBeam)	803.80	27076.50	18.89
8h	Ours (Full)	1261.50	8564.82	0.00
8h	Ours (NoBeam)	1020.28	61897.36	100.00

ciency layers. Negative pruning has a different role and should be argued mainly through supporting tasks. Figure 5 directly visualizes the deltas relative to Full. An additional Pareto-style scatter view is provided in Appendix A (Figure 9).

5.2.3 Supporting Study A: Budget-Sliced View of Beam and Dominance

To evaluate RQ5, we compare the 4h and 8h slices of the same main ablation. The point is not that Beam is always beneficial in exactly the same way, but that its interaction with the budget length is informative.

Table 6 shows a budget-dependent effect. At 4h, removing Beam can buy some extra score (803.80 vs 737.04), but only by paying 35.10× runtime and a non-trivial cap-hit rate. At 8h, the same relaxation no longer helps: `Full` dominates `NoBeam` on both score and runtime, and `NoBeam` hits the cap on all tasks. Therefore, RQ5 is supported in a more precise form: Beam is one of the two core search-space control layers, and its importance becomes especially visible as the budget grows.

5.2.4 Supporting Study B: Negative Pruning for Theme-Constrained Tasks

For RQ6, we use the project-aligned route-theme task rather than the old keyword-only theme benchmark. The study contains 384 paired route-theme cases and compares `Full` vs `NoNeg` under the same starts, departures, and budgets.

At the paired-case level, Full improves theme-hit ratio in 226/384 pairs, with only 2 regressions, and the final selected route changes in 366/384 pairs. The mean deltas are +0.288 in ThemeHit, +0.313 in ThemeSat, +37.89 in RideMinutes, and +12.17 s in runtime. This is the intended role of Negative pruning: it is not the main speedup layer, but it does preserve route-theme fidelity in the project-style task. Supplementary visualizations are provided in Appendix A (Figures 10–11).

Table 7 Supporting route-theme study for Negative pruning.

Method	RideMinutes	ThemeScore	ThemeHit	ThemeSat	Elapsed (ms)
Ours (Full)	318.98	13.81	0.965	0.958	18473.95
Ours (NoNeg)	281.09	5.58	0.677	0.646	6304.29

5.2.5 Interpretation: Why the Three Layers Are All Meaningful

Experiment 2 provides complementary evidence:

- **Dominance pruning** is one of the two core scalability layers; removing it causes a clear search-space blow-up.
- **Beam pruning** is the other core scalability layer; removing it can occasionally buy score at short budgets, but it becomes unstable and expensive as the budget grows.
- **Negative pruning** is a semantic-alignment layer; it improves route-theme preservation, with an explicit runtime trade-off.

Therefore, the three-layer design is justified as a *functional decomposition*: joint search-space control (Beam + Dominance) and objective alignment (Negative).

5.3 Experiment 3: Balance Analysis Supplementing Experiment 1

5.3.1 Objective and Protocol

Experiment 3 is not another independent benchmark. Instead, it supplements Experiment 1 by asking whether the methods remain close to the quality–speed ideal point under the same stop-score task. The goal is to answer three balance-oriented questions:

- **RQ7 (ideal-point closeness):** Is Ours the method closest to the score–time ideal point?
- **RQ8 (trade-off frequency):** How often is Ours simultaneously higher-score and faster than NSGA-II?
- **RQ9 (budget consistency):** Does the balance advantage hold from 2h to 8h?

The current Exp3 analysis is derived from the updated Exp1 source results with 10 origins, 3 departure times, 4 budgets, and 5 stop-score seeds. This yields 600

Table 8 Query-level balance summary of Experiment 3.

Method	Stop Score	Elapsed (ms)	Ideal distance	Avg. balance rank
Ours (Full)	826.90	4165.67	0.1672	1.73
ILS+2opt	697.46	518.79	0.3201	2.50
VNS	645.00	169.31	0.3996	3.53
SA	589.46	130.80	0.5297	4.66
McRAPTOR	546.70	3684.86	0.5489	4.63
NSGA-II	760.66	27351.18	0.6292	5.03
Greedy (L=3)	416.99	3.05	0.7975	6.41
Pure RAPTOR	320.31	56483.31	1.2516	7.48

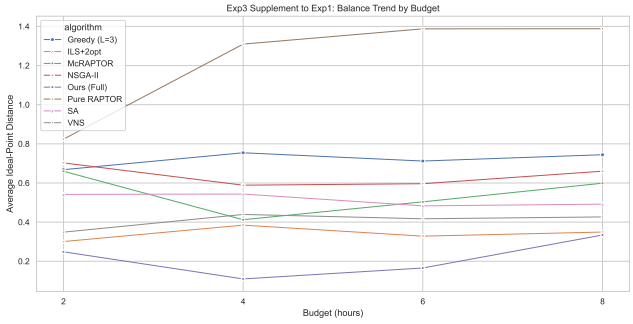


Fig. 6 Experiment 3 as a balance analysis supplement to Experiment 1. The primary metric is `ideal_distance`, shown across the 2h–8h budgets.

task-level rows per algorithm and 120 base-query rows after averaging over stop-score seeds.

5.3.2 Main Balance Results

The first finding is that Ours is the closest method to the score–time ideal point. At the task level, Ours has average ideal distance 0.214, and at the query level it further improves to 0.167, which is the best among all compared methods. NSGA-II, despite being the strongest external baseline on raw score, remains much farther from the ideal point because of its substantially larger runtime.

Figure 6 summarizes the role of Experiment 3 as a balance analysis supplement to Experiment 1 rather than as a separate claim that Ours wins every single task.

5.3.3 Budget-Sliced View

The budget-sliced balance view is also consistent. At 2h, Ours has ideal distance 0.2482 vs 0.7026 for NSGA-II. At 4h, Ours has ideal distance 0.1097 vs 0.5888 for NSGA-II. At 6h, Ours has ideal distance 0.1656 vs 0.5963 for NSGA-II. At 8h, Ours has ideal distance 0.3342 vs 0.6602 for NSGA-II. Thus, the balance advantage is visible throughout the full 2h–8h range, not only in the previously reported 4h and 8h slices.

5.3.4 Ours vs NSGA-II Trade-Off Counts

The second finding is that the ideal-distance advantage is not merely an average effect. At the task level, Ours is both higher-score and faster than NSGA-II in 358/600 tasks. At the query level, the same pattern holds in 82/120 queries. By budget, the query-level counts are 17/30 at 2h, 27/30 at 4h, 19/30 at 6h, and 19/30 at 8h.

5.3.5 Interpretation

Experiment 3 supports three practical claims:

- Ours is the closest method to the score–time ideal point under the current Exp1 task;
- the Exp1 superiority claim is reinforced by the fact that Ours is simultaneously higher-score and faster on many tasks and queries;
- the current balance conclusion is stable across the full 2h–8h budget range, with the clearest lead at 4h and consistent advantage at 6h and 8h.

Therefore, Exp3 is best interpreted as a *supplementary balance analysis* for Experiment 1 rather than as an independent parameter-sensitivity benchmark.

5.4 Experiment 4: Route Case Study

5.4.1 Objective and Positioning

Experiment 4 is not designed as another benchmark. Its purpose is to illustrate the quantitative differences observed in Experiments 1 and 3 through concrete route examples, making the results more interpretable. We address the following questions:

- **RQ10 (success case):** Can the score and speed advantages of Ours be visually confirmed on specific queries?
- **RQ11 (balance case):** Is the quality–speed balance claim reflected in the route structure?
- **RQ12 (limitation):** Are there queries where Ours does not achieve the highest score?

5.4.2 Case Selection Protocol

To avoid arbitrary cherry-picking, the cases are selected from the Experiment 1 results using systematic criteria. We use three cases from the score-guided benchmark:

- **Case A (success):** the query where Ours clearly wins on both stop score and generation time;
- **Case B (balance):** a query where Ours is not the top scorer, but achieves the best quality–speed balance;
- **Case C (counterexample):** a query where Ours is faster but NSGA-II reaches a higher score.

Each case compares Ours against NSGA-II and one additional baseline, with stop score, generation time, score per hour, and scored stops reported per route.

Table 9 Route case study: selected cases from Experiment 1.

Case	Algorithm	Stop score	Gen time (ms)	Score / hour	Scored stops
A	Ours	1459.0	9091.1	185.5	22
A	NSGA-II	854.0	14129.0	108.1	13
A	VNS	508.0	37.0	68.5	10
B	Ours	1276.0	4473.7	164.6	18
B	NSGA-II	1590.0	18956.2	200.8	22
B	SA	1506.0	156.4	192.7	20
C	Ours	1358.0	2730.8	187.3	18
C	NSGA-II	1556.0	20367.1	207.9	19
C	VNS	1461.0	192.6	186.5	19

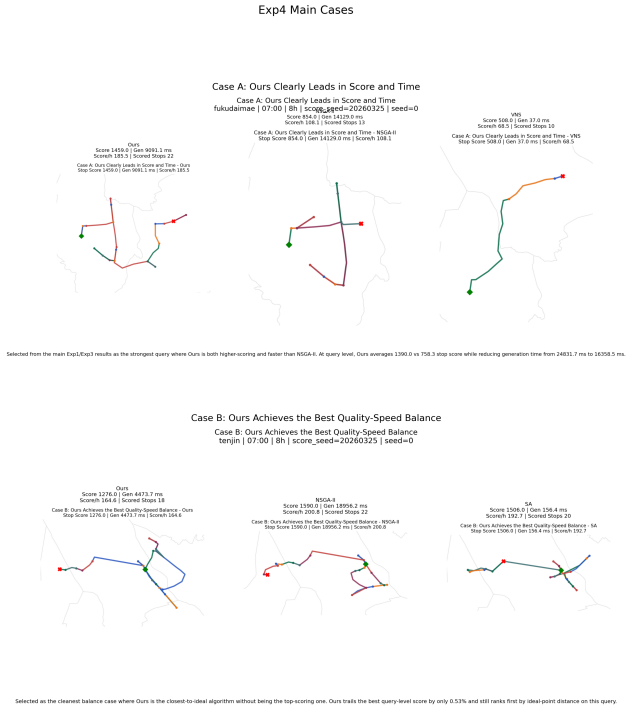


Fig. 7 Route case study main panels. Top: Case A, where Ours clearly leads in both stop score and generation time. Bottom: Case B, where Ours is not the top scorer but achieves the best quality–speed balance.

5.4.3 Main Results

Table 9 summarizes the three cases.

Case A is the clearest success example. Ours achieves a stop score of 1459.0 with a generation time of 9091.1 ms, while NSGA-II reaches only 854.0 in 14129.0 ms. This confirms that there exist queries where Ours simultaneously achieves a substantially higher score and a shorter generation time.

Case B is the intended quality–speed balance example. Here Ours does not achieve the highest score (1276.0 vs 1590.0 for NSGA-II), but its generation time is much shorter (4473.7 ms vs 18956.2 ms). This case visually reinforces the Experiment 3 claim that Ours most often stays close to the score–time ideal point. Figure 7 visualizes the main route-level contrast between the success case and the balance case.

Case C is kept as a counterexample. Ours is faster (2730.8 ms vs 20367.1 ms) but does not reach the highest score (1358.0 vs 1556.0 for NSGA-II). This case is placed in Appendix A (Figure 13) to explicitly avoid claiming that Ours wins on every query.

5.4.4 Interpretation

Experiment 4 provides a route-level interpretability check for the aggregate findings. Case A and Case B visually confirm the quantitative claims of Experiments 1 and 3, while Case C improves credibility by showing a limitation.

6 Discussion

The experimental results support a more precise claim than simply “ReRAPTOR obtains the best routes.” Under the current seeded stop-reward benchmark, the main advantage of ReRAPTOR is that it most consistently balances route quality and computation time. In Experiment 1, it achieves the highest overall average stop score and outperforms NSGA-II by +8.71% on that metric while remaining 6.57× faster on average. Experiment 3 strengthens this interpretation: ReRAPTOR has the best query-level ideal distance (0.1672) and is simultaneously higher-score and faster than NSGA-II in 82/120 base queries. At the same time, the case study in Experiment 4 makes clear that the method is not universally dominant on every individual query, which is important for a credible evaluation.

The ablation results also clarify how the proposed method should be understood. Beam pruning and Dominance pruning are not optional implementation details; they are the two main mechanisms that keep the label search computationally manageable. When Dominance is removed, runtime and label count increase substantially even though the search still finishes. When Beam is removed, the search becomes much more expensive, and the effect is especially severe for the 8-hour budget where cap hits occur in all tasks. When both are removed together, the search space grows too quickly and route quality also degrades. Negative pruning plays a different role. It does not provide the main speedup on the generic stop-reward benchmark, but the theme-constrained supporting study shows that it improves theme-hit ratio and route-theme satisfaction, so its value lies in objective alignment rather than raw scalability.

These findings have two practical implications. First, a timetable-feasible transit itinerary planner does not need to maximize score at any computational cost to be useful; for interactive or repeated query settings, the quality–speed balance is often the more relevant

criterion. Second, the objective module should remain explicit. The same search backbone behaves differently depending on whether the task rewards geometry, stop collection, or thematic coherence, and the experiments show that the pruning layers interact with those objectives in different ways. This argues against treating all value-maximizing transit planning tasks as instances of a single fixed scalar objective.

Several limitations remain. The experiments are based on one city's Ekispert-provided transit schedule data, so cross-city generalization is not yet established. The main benchmark uses seeded stop rewards as a synthetic proxy for value, which is useful for controlled comparison but is still not a substitute for user studies or real preference learning. In addition, some implementation constants, such as beam width, lookahead window, and timeout-related guards, are fixed practical defaults rather than globally optimized settings. Future work should therefore study transfer across cities, learn reward modules from observed user behavior, and investigate adaptive pruning policies that vary with budget length and search difficulty.

7 Conclusion

This paper presented ReRAPTOR, a reverse-objective RAPTOR-style method for time-budgeted value-maximizing transit itinerary planning on timetable-based transit schedules. Its main methodological contribution is to optimize route value during timetable-feasible expansion through a shared search backbone with pluggable objective modules and a three-layer pruning design, rather than scoring candidate routes only after they are fully constructed.

Across the main stop-reward benchmark, ReRAPTOR achieves the best overall quality–speed profile among the compared baselines. It attains the highest average stop score (826.90), exceeds NSGA-II by 8.71% on that metric, and remains 6.57× faster on average. The supplementary balance analysis shows that it is also the closest method to the score–time ideal point, and the ablation study shows that Beam and Dominance are the two core scalability layers, whereas Negative pruning mainly improves semantic alignment in constrained theme tasks. The route case study further confirms both the strengths of the method and the fact that it does not win every query.

Overall, the results indicate that ReRAPTOR is a practical approach for value-oriented transit itinerary planning when the goal is not only to obtain high-scoring routes, but to do so with stable and usable response times. This makes it a strong foundation for future work on preference-aware, theme-aware, and city-scale transit exploration systems.

Acknowledgment

This study used transit schedule data for Fukuoka provided through Ekispert services (Powered by Ekispert API).

References

1. Abuaisha, A., Wallace, M., Harabor, D., Shen, B.: Efficient and exact public transport routing via a transfer connection database. In: Proceedings of the International Symposium on Combinatorial Search, vol. 17, pp. 2–10. Association for the Advancement of Artificial Intelligence (AAAI) (2024). DOI 10.1609/socs.v17i1.31536. URL https://doi.org/10.1609/socs.v17i1.31536
2. Agarwal, P., Rambha, T.: Scalable algorithms for bicriterion trip-based transit routing. IEEE Transactions on Intelligent Transportation Systems **25**(10), 14313–14327 (2024). DOI 10.1109/tits.2024.3391343. URL https://doi.org/10.1109/tits.2024.3391343
3. Ataei, M., Divsalar, A., Saberi, M.: The bi-objective orienteering problem with hotel selection: an integrated text mining optimisation approach. Information Technology and Management **25**(3), 247–275 (2022). DOI 10.1007/s10799-022-00377-5. URL https://doi.org/10.1007/s10799-022-00377-5
4. Choachaicharoenkul, S., Coit, D., Wattanapongsakorn, N.: Multi-objective trip planning with solution ranking based on user preference and restaurant selection. IEEE Access **10**, 10688–10705 (2022). DOI 10.1109/access.2022.3144855. URL https://doi.org/10.1109/access.2022.3144855
5. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation **6**(2), 182–197 (2002). DOI 10.1109/4235.996017. URL https://doi.org/10.1109/4235.996017
6. Delling, D., Dibbelt, J., Pajor, T.: Fast and exact public transit routing with restricted pareto sets. In: 2019 Proceedings of the Twenty-First Workshop on Algorithm Engineering and Experiments (ALENEX), pp. 54–65. Society for Industrial and Applied Mathematics (2019). DOI 10.1137/1.9781611975499.5. URL https://doi.org/10.1137/1.9781611975499.5
7. Delling, D., Dibbelt, J., Pajor, T., Zündorf, T.: Faster transit routing by hyper partitioning. In: 17th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2017), vol. 59, pp. 8:1–8:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2017). DOI 10.4230/oasics.atmos.2017.8. URL https://doi.org/10.4230/oasics.atmos.2017.8
8. Delling, D., Pajor, T., Werneck, R.F.: Round-based public transit routing. Transportation Science **49**(3), 591–604 (2015). DOI 10.1287/trsc.2014.0534. URL https://doi.org/10.1287/trsc.2014.0534
9. Demeyer, S., Goedgebeur, J., Audenaert, P., Pickavet, M., Demeester, P.: Speeding up martins' algorithm for multiple objective shortest path problems. 4OR **11**(4), 323–348 (2013). DOI 10.1007/s10288-013-0232-5. URL https://doi.org/10.1007/s10288-013-0232-5

10. Divsalar, G., Divsalar, A., Jabbarzadeh, A., Sahebi, H.: An optimization approach for green tourist trip design. Soft Computing **26**(9), 4303–4332 (2022). DOI 10.1007/s00500-022-06834-1. URL https://doi.org/10.1007/s00500-022-06834-1
11. Euler, R., Lindner, N., Borndörfer, R.: Price optimal routing in public transportation. EURO Journal on Transportation and Logistics **13**, 100128 (2024). DOI 10.1016/j.ejtl.2024.100128. URL https://doi.org/10.1016/j.ejtl.2024.100128
12. Fan, L., Mumford, C.L., Evans, D.: A simple multi-objective optimization algorithm for the urban transit routing problem. In: 2009 IEEE Congress on Evolutionary Computation, pp. 1–7. IEEE (2009). DOI 10.1109/cec.2009.4982923. URL https://doi.org/10.1109/cec.2009.4982923
13. Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., Vathis, N.: Heuristics for the time dependent team orienteering problem: Application to tourist route planning. Computers & Operations Research **62**, 36–50 (2015). DOI 10.1016/j.cor.2015.03.016. URL https://doi.org/10.1016/j.cor.2015.03.016
14. Godfrid, J., Radnic, P., Vaisman, A., Zimányi, E.: Analyzing public transport in the city of buenos aires with mobilitydb. Public Transport **14**(2), 287–321 (2022). DOI 10.1007/s12469-022-00290-8. URL https://doi.org/10.1007/s12469-022-00290-8
15. González-Navasa, C., Moreno-Pérez, J.A., Brito-Santana, J., Alonso-Afonso, H.: Recommendation of tourist itineraries with dependence on transport time. Transportation Research Procedia **71**, 77–84 (2023). DOI 10.1016/j.trpro.2023.11.060. URL https://doi.org/10.1016/j.trpro.2023.11.060
16. Gunawan, A., Lau, H.C., Vansteenwegen, P.: Orienteering problem: A survey of recent variants, solution approaches and applications. European Journal of Operational Research **255**(2), 315–332 (2016). DOI 10.1016/j.ejor.2016.04.059. URL https://doi.org/10.1016/j.ejor.2016.04.059
17. He, P., Jiang, G., Lam, S.K., Sun, Y., Ning, F.: Exploring public transport transfer opportunities for pareto search of multicriteria journeys. IEEE Transactions on Intelligent Transportation Systems **23**(12), 22895–22908 (2022). DOI 10.1109/tits.2022.3194523. URL https://doi.org/10.1109/tits.2022.3194523
18. Işık, E.E., Ayyıldız, E., Taşkın, A.: A constraint programming approach for multi-objective tourist trip design problem with mandatory visits: A case study for izmir turkey. Journal of Project Management **9**(1), 61–72 (2024). DOI 10.5267/j.jpm.2023.8.003. URL https://doi.org/10.5267/j.jpm.2023.8.003
19. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983). DOI 10.1126/science.220.4598.671. URL https://doi.org/10.1126/science.220.4598.671
20. Kujala, R., Weckström, C., Darst, R.K., Mladenović, M.N., Saramäki, J.: A collection of public transport network data sets for 25 cities. Scientific Data **5**(1) (2018). DOI 10.1038/sdata.2018.89. URL https://doi.org/10.1038/sdata.2018.89
21. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search. In: F. Glover, G.A. Kochenberger (eds.) Handbook of Metaheuristics, *International Series in Operations Research & Management Science*, vol. 57, pp. 321–353. Kluwer Academic Publishers (2003). DOI 10.1007/0-306-48056-5_11. URL https://doi.org/10.1007/0-306-48056-5_11
22. Luo, X.G., Liu, X.R., Ji, P.L., Shang, X.Z., Zhang, Z.L.: Trip planning for visitors in a service system with capacity constraints. Computers & Operations Research **148**, 105974 (2022). DOI 10.1016/j.cor.2022.105974. URL https://doi.org/10.1016/j.cor.2022.105974
23. Marzal, E., Sebastiá, L.: Solving the tourist trip design problem with time windows and variable profit using incremental local search. Applied Soft Computing **155**, 111399 (2024). DOI 10.1016/j.asoc.2024.111399. URL https://doi.org/10.1016/j.asoc.2024.111399
24. Mladenović, N., Hansen, P.: Variable neighborhood search. Computers & Operations Research **24**(11), 1097–1100 (1997). DOI 10.1016/S0305-0548(97)00031-2. URL https://doi.org/10.1016/S0305-0548(97)00031-2
25. Pereira, R.H.M., Andrade, P.R., Vieira, J.P.B.: Exploring the time geography of public transport networks with the gtfs2gps package. Journal of Geographical Systems **25**(3), 453–466 (2022). DOI 10.1007/s10109-022-00400-x. URL https://doi.org/10.1007/s10109-022-00400-x
26. Persia, F., Pilato, G., Ge, M., Bolzoni, P., D'Auria, D., Helmer, S.: Improving orienteering-based tourist trip planning with social sensing. Future Generation Computer Systems **110**, 931–945 (2020). DOI 10.1016/j.future.2019.10.028. URL https://doi.org/10.1016/j.future.2019.10.028
27. Phan, D.M., Viennot, L.: Fast public transit routing with unrestricted walking through hub labeling. In: Analysis of Experimental Algorithms, pp. 237–247. Springer International Publishing (2019). DOI 10.1007/978-3-030-34029-2_16. URL https://doi.org/10.1007/978-3-030-34029-2_16
28. Pilato, G., Persia, F., Ge, M., Chondrogiannis, T., D'Auria, D.: A modular social sensing system for personalized orienteering in the covid-19 era. ACM Transactions on Management Information Systems **14**(4), 1–26 (2023). DOI 10.1145/3615359. URL https://doi.org/10.1145/3615359
29. Ruiz-Meza, J., Brito, J., Montoya-Torres, J.R.: A grasp to solve the multi-constraints multi-modal team orienteering problem with time windows for groups with heterogeneous preferences. Computers & Industrial Engineering **162**, 107776 (2021). DOI 10.1016/j.cie.2021.107776. URL https://doi.org/10.1016/j.cie.2021.107776
30. Ruiz-Meza, J., Brito, J., Montoya-Torres, J.R., Castro-Vergara, A.: Green fuzzy tourist trip design problem. Advances in Operations Research **2022**, 1–10 (2022). DOI 10.1155/2022/6828385. URL https://doi.org/10.1155/2022/6828385
31. Ruiz-Meza, J., Montoya-Torres, J.R.: A systematic literature review for the tourist trip design problem: Extensions, solution techniques and future research lines. Operations Research Perspectives **9**, 100228 (2022). DOI 10.1016/j.orp.2022.100228. URL https://doi.org/10.1016/j.orp.2022.100228
32. Souffriau, W., Vansteenwegen, P.: Tourist trip planning functionalities: State-of-the-art and future. In: Current Trends in Web Engineering, pp. 474–485. Springer Berlin Heidelberg (2010). DOI 10.1007/978-3-642-16985-4_46. URL https://doi.org/10.1007/978-3-642-16985-4_46
33. Souffriau, W., Vansteenwegen, P., Vertommen, J., Vanden Berghe, G., Van Oudheusden, D.: A personalized tourist trip design algorithm for mobile tourist guides. Applied Artificial Intelligence **22**(10), 964–985 (2008). DOI 10.1080/08839510802379626. URL https://doi.org/10.1080/08839510802379626
34. Sylejmani, K., Abdurrahmani, V., Ahmeti, A., Gashi, E.: Solving the tourist trip planning problem with attraction

patterns using meta-heuristic techniques. Information Technology & Tourism **26**(4), 633–678 (2024). DOI 10.1007/s40558-024-00297-w. URL `https://doi.org/10.1007/s40558-024-00297-w`

35. Tenemaza, M., Luján-Mora, S., de Antonio, Á., Ramírez, J.: Improving itinerary recommendations for tourists through metaheuristic algorithms: An optimization proposal. IEEE Access **8**, 79003–79023 (2020). DOI 10.1109/access.2020.2990348. URL `https://doi.org/10.1109/access.2020.2990348`
36. Weng, D., Chen, R., Zhang, J., Bao, J., Zheng, Y., Wu, Y.: Pareto-optimal transit route planning with multi-objective monte-carlo tree search. IEEE Transactions on Intelligent Transportation Systems **22**(2), 1185–1195 (2021). DOI 10.1109/tits.2020.2964012. URL `https://doi.org/10.1109/tits.2020.2964012`
37. Wessel, N., Allen, J., Farber, S.: Constructing a routable retrospective transit timetable from a real-time vehicle location feed and gtfs. Journal of Transport Geography **62**, 92–97 (2017). DOI 10.1016/j.jtrangeo.2017.04.012. URL `https://doi.org/10.1016/j.jtrangeo.2017.04.012`
38. Wong, J.: Leveraging the general transit feed specification for efficient transit analysis. Transportation Research Record: Journal of the Transportation Research Board **2338**(1), 11–19 (2013). DOI 10.3141/2338-02. URL `https://doi.org/10.3141/2338-02`

A Appendix A: Supplementary Visualizations (Exp1–Exp4)

This appendix collects the supplementary figures referenced in the main text. Their purpose is to make the main claims more transparent rather than to add a new evaluation story. Appendix figures for Experiment 1 show robustness across random stop-score seeds, those for Experiment 2 visualize the trade-offs behind the ablation and theme-support studies, the Experiment 3 figure complements the ideal-distance summary with a quality–speed scatter view, and the Experiment 4 appendix keeps an explicit counterexample to avoid overclaiming.

A.1 Experiment 1 Supplementary Figures

The figure in this subsection reports seed-level stability for the updated seeded stop-reward benchmark. It supports the main-text statement that the advantage of ReRAPTOR over NSGA-II remains positive across the tested score seeds, even though the gap varies by seed and budget.

A.2 Experiment 2 Supplementary Figures

These figures provide additional evidence for the role separation observed in the ablation study. The Pareto-style scatter highlights the efficiency frontier of the core pruning variants, while the remaining figures summarize how Negative pruning changes route-theme behavior in the supporting study at the aggregate level.

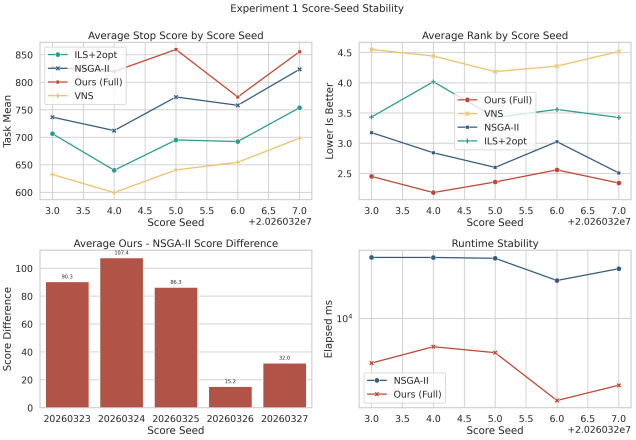


Fig. 8 Supplementary to Experiment 1: score-seed stability under the seeded stop-reward benchmark.

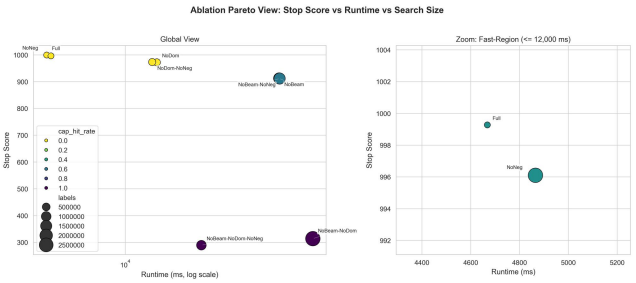


Fig. 9 Supplementary to Experiment 2: Pareto-style ablation scatter view.

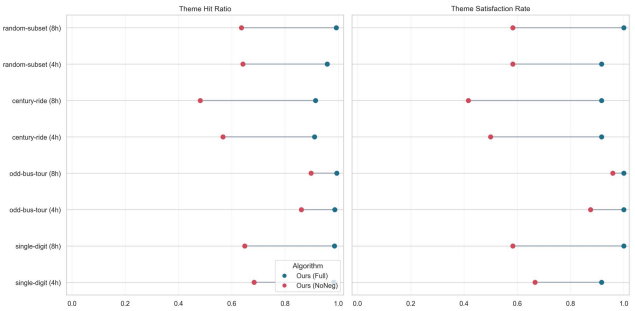


Fig. 10 Supplementary to Experiment 2: paired route-theme gains of `Full` over `NoNeg`.

A.3 Experiment 3 Supplementary Figures

The following scatter plot complements the main ideal-distance figure by showing how the methods are distributed in the quality–speed plane. It helps explain why ReRAPTOR is closest to the ideal point even though some baselines are occasionally faster or occasionally higher-scoring on individual queries. Figure 12 provides this supplementary quality–speed view.

A.4 Experiment 4 Supplementary Figures

The appendix case for Experiment 4 is intentionally retained as a limitation example. It shows a query where ReRAPTOR

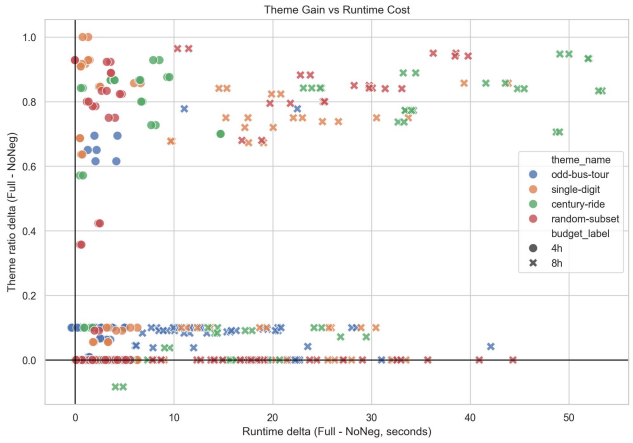


Fig. 11 Supplementary to Experiment 2: trade-off view of route-theme preservation and runtime overhead.

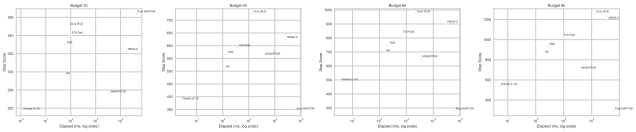


Fig. 12 Supplementary to Experiment 3: quality–speed scatter around the ideal-point balance view.

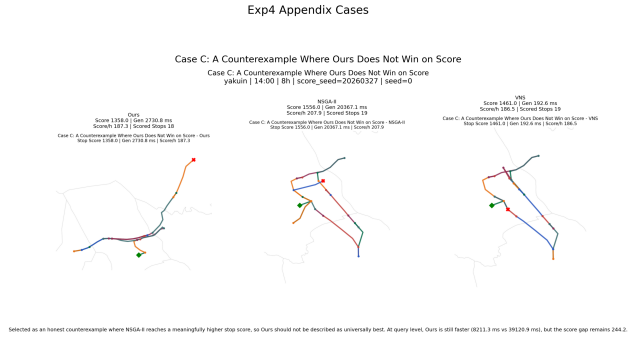


Fig. 13 Supplementary to Experiment 4: Case C, kept as a counterexample where Ours is faster but does not reach the highest stop score.

is substantially faster than NSGA-II but does not achieve the highest stop score, reinforcing that the contribution of the method is overall balance rather than universal per-query dominance. Figure 13 shows this explicit counterexample.